

WRRR Game 1 0

복잡게임의 최소 충분상태와 실제 바둑 엔진

오목과 바둑의 구성적 분석 완전탐색 검증 및 WRRR Go 0 1

Wonsik Choi 최원식

Independent Researcher

janefather@gmail.com

2026년 9월 17일

WRRR Core 1.0 동결 유지

Zenodo 공개용 통합 연구보고서와 재현 패키지

연구 결론

WRRR Game 연구는 하나의 고정된 실행문법을 오목형 연결게임과 바둑에 적용해 세 단계를 완성했다. 0.1은 강제승리와 패의 구성적 사례를 계산했고, 0.2는 두 축소 규칙계의 전체 도달상태를 열거해 Renderer의 값 보존과 residue의 필수성을 검증했으며, 1.0은 같은 상태와 전이 구조 위에 실제 착수를 선택하는 WRRR Go 0.1 엔진을 구현했다. 따라서 현재 결과는 개념 대응, 전수검증, 실행 가능한 의사결정 알고리즘을 하나의 계보로 연결한다.

단계	질문	실행 결과	확정된 의미
Game 0.1	구조가 실제 사례를 계산하는가	9×9 오목 70응답과 5×5 패 반례	구성적 작동 가능성
Game 0.2	값과 합법성이 전 상태에서 보존되는가	6,036,001 Connect K 상태와 132,161 바둑 상태	유한 규칙계의 전수검증
Game 1.0	같은 구조로 실제 착수를 선택할 수 있는가	9×9 대국 가능 엔진과 색상 교대 벤치마크	계산 구조에서 행위자로 확장

가장 중요한 용어도 바로잡았다. residue는 미래를 예측하거나 보존하지 않는다. residue는 현재 판만으로 판정할 수 없는 합법수 제약을 현재 상태에 포함해 합법수 집합과 다음 상태의 갱신 규칙을 정확히 만든다. 탐색 역시 이미 존재하는 미래를 알아내는 과정이 아니라, 현재 규칙이 허용하는 후속 상태를 생성하고 목표함수에 따라 비교하는 계산이다.

핵심 수치

영역	전수 또는 대국 범위	결과	판정
Connect K	도달상태 6,036,001개	minimax 불일치 0개	안전축약 값 보존
Connect K	빈 판 alpha beta	245,560 → 32노드	같은 root 값
3×3 바둑	도달상태 132,161개	residue 의존 동치류 752개	board only 불충분
WRR Go	균등 MCTS 상대 16국	15승 1패	같은 simulation 수
WRR Go	무작위 상대 16국	16승 0패	색상 교대
9×9 자기대국	82수	W 승 흑 기준 -8.5집	two_passes

대국 수치는 전문 바둑엔진과의 기력 비교가 아니라, WRR Renderer가 실제 합법수 생성과 수 선택에 연결되고 동일 계산량의 통제 알고리즘과 비교 가능한지를 확인한 첫 실행시험이다. 모든 대국은 색을 교대했고 같은 판 크기, 같은 simulation 수, 같은 득점 규칙을 사용했다.

문서 구성

1. WRR Game 공통 실행계약과 예측으로 오인될 표현의 정정
2. Game 0.1의 오목 강제승리와 바둑 패 구성적 분석
3. Game 0.2의 Connect K 및 3×3 바둑 완전탐색
4. WRR Go 0.1의 상태, Renderer, MCTS 결합과 실제 코드
5. 규칙검사, 대조 대국, 9×9 자기대국 및 재현 장부
6. 학문적 가치, 적용 범위, 후속 개발 기준과 Zenodo 메타데이터

1 공통 실행계약

WRR Core 1 0을 고치지 않는 응용

게임은 WRR Core 1.0의 수정안이 아니라 Domain Profile이다. Core는 어떤 게임을 특별 취급하지 않고 SOURCE, LAW, STATE와 RESIDUE, BOUNDARY, COMMON CARRIER, UPDATE, RENDERER, PHENOTYPE, LEDGER의 실행 순서만 고정한다. 각 게임의 규칙과 목표는 Domain Profile에 들어간다. 이 분리는 오목과 바둑의 차이를 지우지 않으면서 같은 분석 질문을 던지게 한다.

Core 항목	오목형 연결게임	바둑	바둑 엔진
SOURCE	빈 칸의 착수	착수 또는 패스	사람 또는 알고리즘의 행동
LAW	교대와 연속 K개	활로 포획 자살금지 단순 패	같은 규칙의 실행 함수
STATE	판과 차례	판 차례 residue 패스 수	수 번호와 탐색노드 추가
RESIDUE	추가 이력 0	직전 판 1개	패 판정에 그대로 사용
BOUNDARY	빈 칸과 승리선	합법 착수 경계	합법 후보만 탐색
CARRIER	유한 격자	유한 직교 격자	동일 좌표와 점유 배열
UPDATE	돌 추가와 차례교대	포획 패 판정 후 교대	불변 GoState 반환
RENDERER	승리 위협 강제방어	포획 활로 연결 절단	prior와 rollout 순서
PHENOTYPE	승리 강제패배 열린탐색	capture rescue atari 등	착수 설명 라벨
LEDGER	상태 값 노드	합법수와 반례 이력	후보 방문 가치 대국기록

상태충분성

상태표현 M이 충분하려면 서로 다른 두 합법 이력이 같은 M으로 압축될 때 합법행동 집합이 같아야 한다. 같은 판 모양을 보고도 이력에 따라 허용되는 수가 달라진다면, M에는 현재의 다음 계산에 필요한 정보가 빠져 있다. 이것이 residue를 추가하는 기준이며 과거 전체를 무조건 저장하는 기준이 아니다.

$$M(h_1) = M(h_2) \Rightarrow A_{legal}(h_1) = A_{legal}(h_2)$$

자유형 오목은 판과 차례로 다음 합법 착수를 정할 수 있다. 단순 패 바둑은 직전 판 하나를 더 알아야 즉시 반복을 금지할 수 있다. positional superko처럼 더 긴 반복을 금지하는 규칙을 선택하면 residue도 그 규칙에 맞게 커져야 한다. 최소기억의 크기는 취향이 아니라 Law가 정한다.

해석 탐색 예측의 구분

용어	이 연구에서의 뜻	쓰지 않는 뜻
상태전이	합법행동을 적용해 다음 상태를 계산	현실의 미래를 미리 관측
후속 분기	현재 상태에서 규칙상 생성 가능한 후보	실제로 반드시 일어날 사건
탐색	후보 상태를 목적값으로 비교	경험적 예측의 자동 성립
residue 보존	합법수 판정에 필요한 제약정보 유지	미래 자체의 저장
Renderer 평가	현재 후보의 구조적 특징과 탐색 우선순위	바둑 결과의 무조건적 예언

따라서 이전 문서의 미래분기 보존이라는 표현은 사용하지 않는다. 정확한 문장은 다음과 같다. residue는 합법수 판정과 다음 상태의 전이 규칙을 정확히 보존한다. Renderer와 MCTS는 규칙이 허용한 후속 상태들을 계산해 행동을 고른다.

2 WRRRA Game 0 1 구성적 분석

9×9 자유형 오목의 강제승리 출구

0.1은 9×9 자유형 오목의 특정 합법 판에서 후보 71개를 모두 조사했다. 중앙 4 4 한 수만 다음 차례의 즉시승리 출구 네 개를 만들었다. 그 뒤 백의 가능한 응답 70개를 전부 적용했지만 네 출구를 동시에 없애는 응답은 없었다. 이 계산은 Renderer가 살아 있는 경계와 다중 위협을 이용해 후보를 줄이는 구성적 사례다.

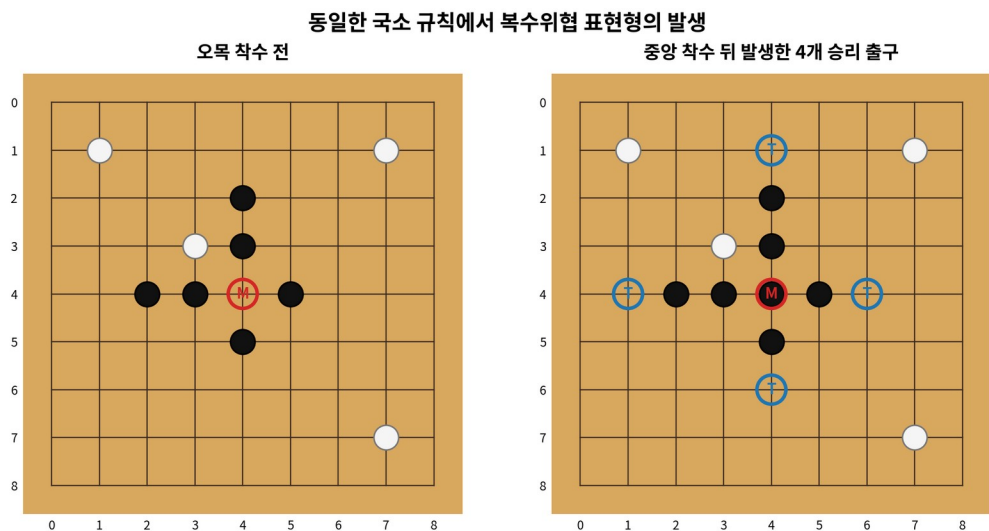


그림 1 9×9 자유형 오목에서 중앙 착수가 만든 네 개의 다음 수 승리 출구

항목	값	의미
합법 후보	71	모든 빈 칸
다중출구 후보	1	중앙 4 4 한 수
만든 승리 출구	4	가로 세로 네 끝점
검사한 방어	70	중앙 뒤의 모든 백 응답
모든 출구를 지운 방어	0	강제승리 판정
후보 감소율	98.59%	구성적 사례의 Renderer 축약

5×5 바둑의 패와 residue

바둑 사례는 현재 판의 모양만으로는 합법수를 정할 수 없다는 직접 반례를 구성했다. 한 수가 백돌 하나를 잡은 뒤 즉시 되따내면 직전 판이 복원된다. 직전 판 residue를 포함하면 합법행동은 18개이고, 이를 제거하면 금지되어야 할 되따내기가 들어와 19개가 된다.

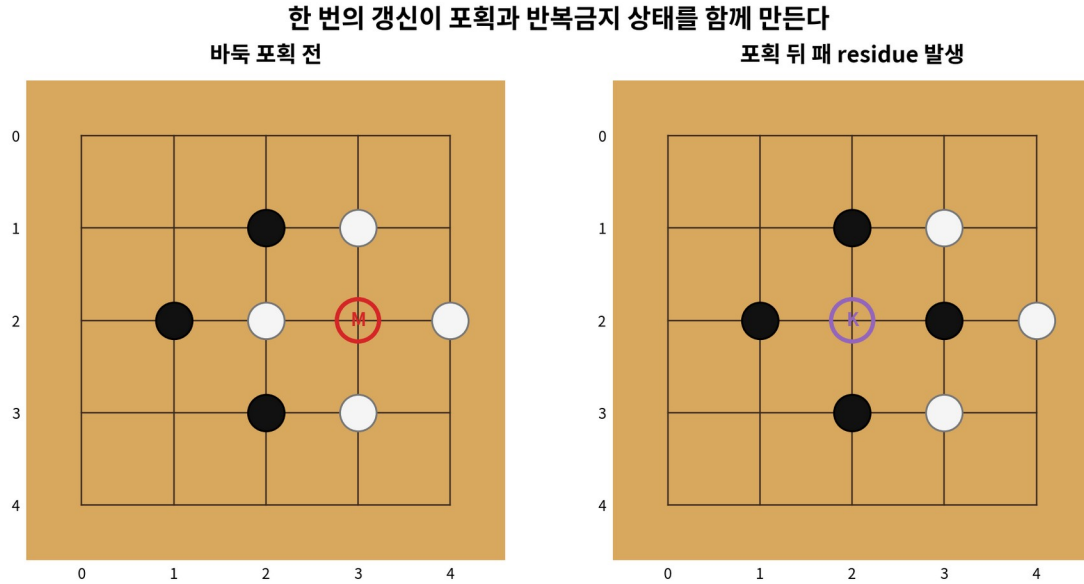


그림 2 단순 패에서 직전 판 residue가 차단하는 즉시 되따내기

검사	residue 포함	residue 제거	판정
되따내기	불법	합법	제거 시 직전 판 복원
합법행동 수	18	19	한 수 차이

0.1의 가치는 두 용어를 실제 계산에 연결했다는 점이다. Renderer는 살아 있는 위협과 경계를 드러내 후보를 정렬하거나 줄인다. residue는 현재 판만으로 보이지 않는 법적 제약을 상태에 결합한다. 0.2는 이 구성적 사례가 전체 도달 상태에서도 유지되는지를 검사했다.

3 WRRRA Game 0 2 완전탐색

4×4 3목 Connect K 실험계

표준 15×15 오목 전체를 열거하는 대신 4×4에서 연속 3개가 승리인 완전열거 실험계를 사용했다. 빈 판에서 시작해 턴 순서와 조기 종료를 지키며 도달 가능한 상태만 단았다. 목적은 작은 판의 기력을 주장하는 것이 아니라 Renderer의 안전축약이 minimax 값을 바꾸는지 모든 상태에서 판정하는 것이다.

장부	값	장부	값
도달상태	6,036,001	합법 방향전이	23,453,344
승리 종료	2,572,442	무승부 종료	18
즉시승리 상태	3,143,087	단일 강제방어	65,828
다중위협 강제패배	232,299	열린 탐색	22,327
전체 값 불일치	0	빈 판 값	1

$$\sum_{S \in S_{reach}} \mathbf{1}[V_{base}(S) \neq V_{WRRRA}(S)] = 0$$

안전축약은 즉시승리 수가 있으면 그 수만 남기고, 상대의 즉시승리 지점이 하나면 그 지점만 막으며, 서로 다른 즉시 승리 지점이 둘 이상이면 한 수로 모두 막을 수 없다는 증명서를 사용한다. 이 조건이 없는 상태에서는 수를 버리지 않고 순서만 바꾼다. 기존 minimax와 이 solver를 6,036,001개 상태 모두에서 비교한 결과 값 불일치는 0개였다.

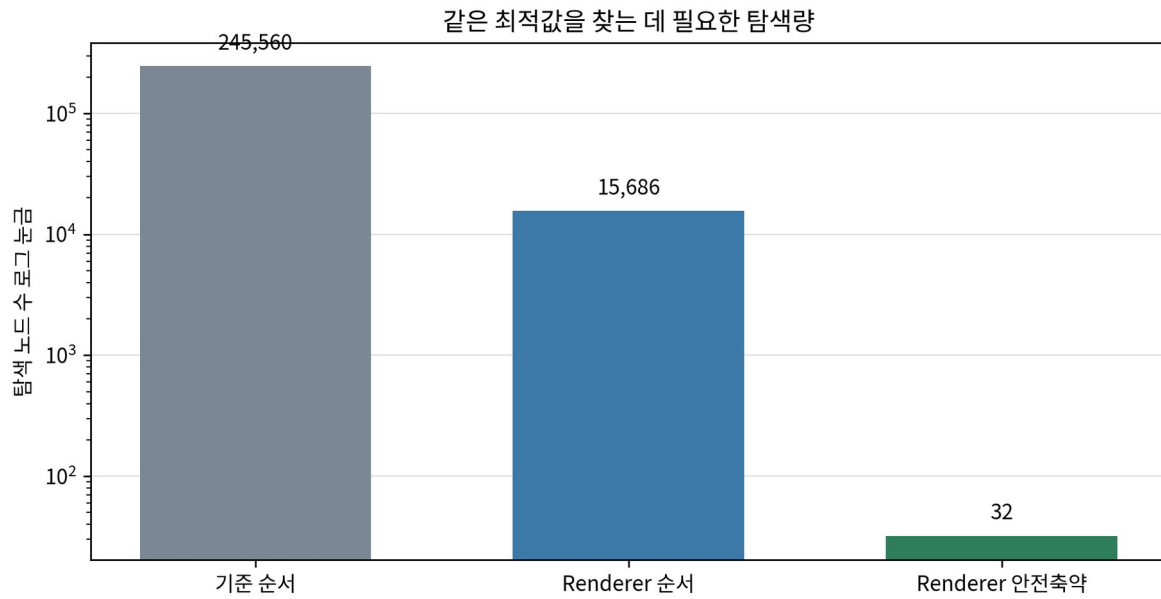


그림 3 같은 빈 판 값을 얻기 위한 alpha beta 방문 노드 수

탐색	방문 노드	기준 대비 감소	root 값
기준 행 우선	245,560	0%	1
Renderer 순서	15,686	93.612%	1
Renderer 안전축약	32	99.98697%	1

3×3 단순 패 바둑의 전체 도달그래프

포획, 자살수 금지, 단순 패, 패스, 두 번의 연속 패스 종료룰 적용해 초기 빈 판에서 도달할 수 있는 완전상태를 모두 열거했다. 완전상태는 현재 판, 차례, 직전 판 residue, 연속 패스 수다. 현재 판, 차례, 패스 수가 같은 상태를 묶은 뒤 residue만 달라질 때 합법 착수 집합이 달라지는지 비교했다.

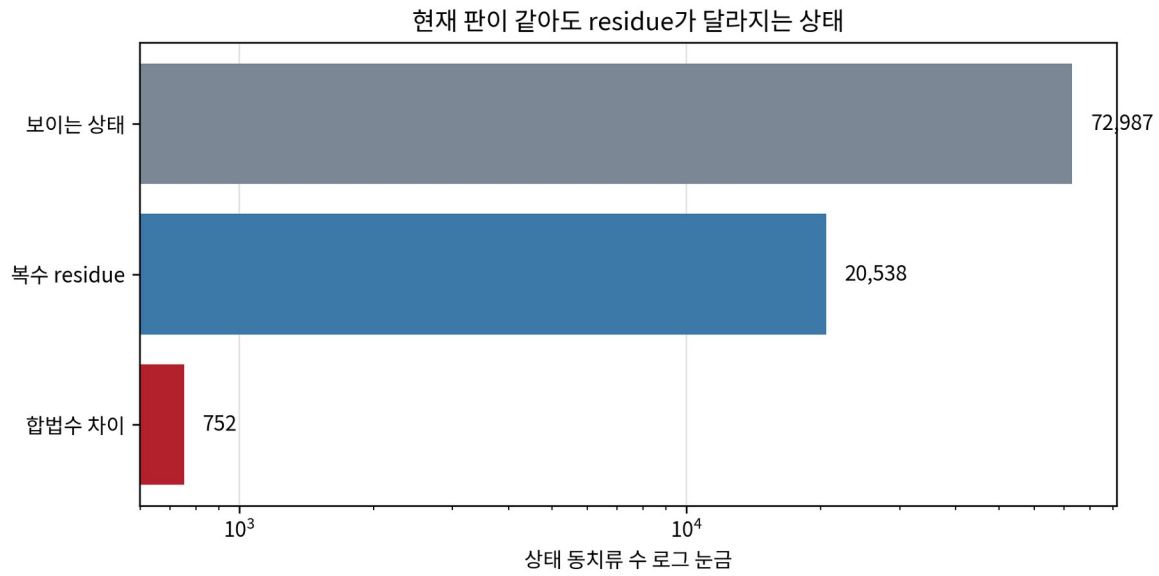


그림 4 같은 가시적 상태 안에서 residue와 합법수 집합이 달라지는 동치류

분류	개수	의미
완전상태	132,161	residue를 포함한 도달상태
합법 방향전이	420,710	착수와 패스
가시적 상태 동치류	72,987	판 차례 패스 수 동일
복수 residue 동치류	20,538	서로 다른 직전 판
합법수 차이 동치류	752	residue만으로 합법수 집합 변화
패 차단 완전상태	768	각 상태에서 정확히 한 착수 차단
최대 합법수 집합 수	3	한 가시적 상태에 대응

백의 0 1 수는 현재 판만으로 판정할 수 없다

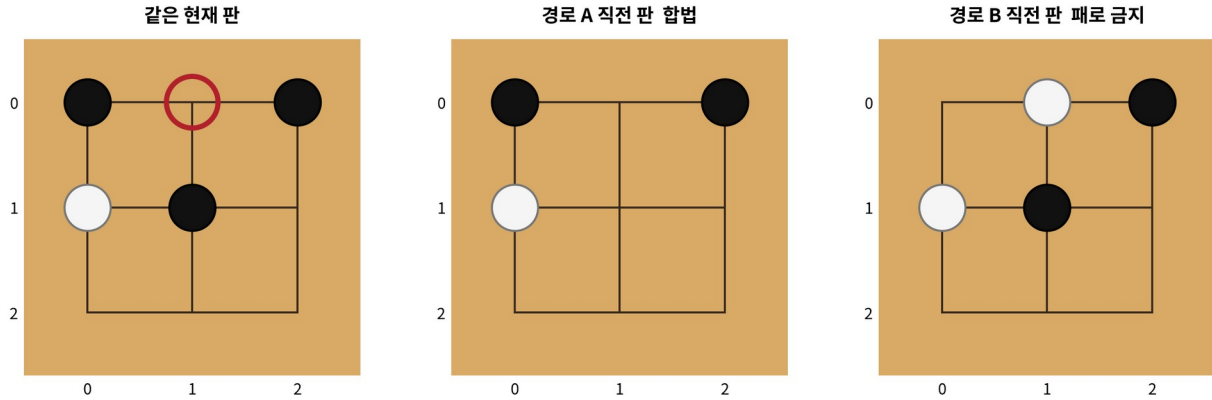


그림 5 같은 현재 판에서 residue에 따라 백의 0 1 합법성이 달라지는 최단 5수 증명쌍

이 결과는 예측 성능에 관한 것이 아니다. 같은 현재 판에서 어떤 수가 규칙상 허용되는지를 정확히 판정하려면 무엇을 상태에 포함해야 하는지에 관한 결과다. 가장 짧은 반례의 두 이력은 모두 5수 뒤 같은 판과 같은 차례에 도달하지만, 한쪽의 백 0 1은 합법이고 다른 쪽에서는 직전 판을 복원하므로 금지된다.

완전탐색이 만든 설계 규칙

제거시험	관측된 변화	설계 판정
Renderer 순서 제거	15,686 → 245,560노드	정확도와 계산비용을 분리
안전축약 제거	32 → 245,560노드	증명된 phenotype은 후보축약 가능
직전 판 residue 제거	768개 상태에서 금지수 1개 유입	합법수 판정과 상태전이에 필수
전체 이력 대신 직전 판	단순 패 판정 유지	Law가 요구한 최소기억

4 WRRR Go 0.1 실제 대국 알고리즘

완전탐색에서 실제 착수로

3×3 전체 열거는 상태정의와 규칙 구현을 검증하지만 9×9 이상에서는 전체 상태공간을 닫을 수 없다. WRRR Go 0.1은 검증된 상태전이를 그대로 사용하고, Renderer가 각 합법수의 구조적 특징을 수치화한 뒤 MCTS가 제한된 계산예산 안에서 후보를 반복 평가하도록 만들었다. 결과적으로 같은 프로그램이 5×5 시험대국과 9×9 사람 대 엔진 대국을 실행한다.

WRRR Go 0.1의 한 수 결정 과정



결과는 미래 예측이 아니라 현재 규칙에서 허용되는 후속 상태의 탐색과 가치평가이다

그림 6 WRRR Go 0.1의 상태에서 착수와 Ledger까지

완전상태와 합법전이

$$S_t = (B_t, p_t, \rho_t, c_t, n_t), \quad \rho_t = B_{t-1}$$

기호	구성	필요한 이유
B	현재 흑백 점유 배열	돌무리 할로 포획 득점
p	돌 차례	행동 주체와 상대를 구분
ρ	직전 판 배열	단순 패의 즉시 반복 금지
c	연속 패스 수	두 패스 종료 판정
n	수 번호	실행 장부와 운영상 탐색 경계

$$T(S_t, a) = S_{t+1} \quad \text{iff} \quad a \in A_{\text{legal}}(S_t; L_D, \rho_t)$$

착수 함수는 빈 칸인지 확인하고, 인접한 상대 돌무리의 활로가 0이면 포획하며, 새로 놓인 자기 돌무리의 활로가 0이면 자살수로 거부한다. 결과 판이 ρ 와 같으면 단순 패로 거부한다. 패스는 판을 바꾸지 않되 직전 판과 연속 패스 수를 갱신한다. 두 번의 연속 패스에서 area score를 계산한다.

투명한 Renderer 채널

Renderer는 신경망의 숨은 파라미터 대신 열한 개의 명시적 채널을 사용한다. 가중치는 학습하지 않고 고정했으며, 각 수의 점수와 phenotype을 Ledger에 남긴다. 이 점수는 합법성을 정하지 않는다. 합법수 함수가 먼저 후보를 담은 뒤 탐색 prior와 rollout 순서에만 사용한다.

$$R(a \mid S) = \sum_{i=1}^{11} w_i f_i(S, a)$$

채널	계산 내용	가중치
capture	즉시 잡은 상대 돌 수	+3.20
rescue	단수였던 인접 자기 돌의 구조적 구제	+2.40
atari	착수 뒤 단수가 된 상대 돌 수	+1.35
connection	돌 이상의 자기 돌무리 연결	+0.72
cut pressure	돌 이상의 상대 돌무리에 동시에 접촉	+0.42
liberties	새 돌무리의 활로 최대 6	+0.20
self atari	포획 없는 한 집 활로	-3.25
eye fill	포획 없이 자기 눈 후보를 메움	-2.10
area delta	작은 폐쇄영역의 즉시 변화 열린 대영역 제외	+0.32

채널	계산 내용	가중치
position	판 크기에 따른 초기 위치 선호	+0.28
pass readiness	상대 패스와 현재 우세를 이용한 종료	+1.00

MCTS 결합

MCTS 자체는 알려진 탐색 알고리즘이다. WRRRA의 역할은 이를 새 이름으로 바꾸는 것이 아니라 충분한 상태, 합법전이, Renderer prior, guided rollout, phenotype과 Ledger를 한 실행계약 안에서 제공하는 데 있다. 같은 simulation 수의 균등 MCTS를 통제군으로 두어 Renderer가 추가한 차이를 직접 비교할 수 있다.

$$a^* = \arg \max_a \left(Q(S, a) + c P_R(a | S) \frac{\sqrt{N(S)}}{1 + N(S, a)} \right)$$

1. 현재 완전상태에서 착수와 패스를 포함한 합법전이를 생성한다.
2. Renderer 점수를 softmax로 정규화해 후보 prior를 만든다.
3. 방문가치 Q와 탐색항을 합친 기준으로 트리를 내려간다.
4. 새 노드에서는 Renderer 상위 후보를 중심으로 제한된 rollout을 실행한다.
5. 종료 또는 계산 경계의 area score를 root 플레이어 관점의 값으로 되돌린다.
6. 가장 많이 방문한 수를 선택하고 상위 후보의 방문 수 가치 prior phenotype을 기록한다.

실제로 사용하는 방법

목적	명령	결과
9×9 사람 대 엔진	python wrra_go_0_1.py play --size 9 --human B	좌표 입력형 대국
규칙 회귀검사	python wrra_go_0_1.py test	패 포획 자살수 종료 검사
색상 교대 벤치마크	python wrra_go_0_1.py benchmark	JSON 성능 장부
9×9 자기대국	python wrra_go_0_1.py selfplay -- size 9	JSON과 SGF 대국기보

사람은 D4 같은 바둑 좌표와 pass를 입력한다. l열은 통상적인 바둑 좌표처럼 건너뛴다. 기본 72 simulations는 실행 가능성을 위한 설정이며 더 높은 수를 주면 한 수당 계산시간을 늘려 더 많은 후보를 검사한다. 13×13과 19×19도 규칙상 실행할 수 있지만 이번 버전의 정량시험은 5×5와 9×9에 집중했다.

5 실행검증

결정적 규칙검사

검사항목	결과
같은 현재 판 증명쌍 재현	PASS
단순 패가 직전 판 복원을 차단	PASS
residue 제거 시 반복수가 유입	PASS
같은 판에서 residue가 합법성 변경	PASS
두 번의 연속 패스 종료	PASS
포획 시 돌무리 제거	PASS
자살수 금지	PASS
area score 결정성	PASS
초반 열린 빈 영역의 영토 오인 방지	PASS
좌표 변환 양복	PASS

이 가운데 핵심 회귀검사는 Game 0.2가 찾은 최단 5수 반례를 새 엔진의 독립된 상태객체로 다시 재생하는 것이다. 두 이력은 같은 판과 같은 차례에 도달하지만 직전 판이 다르다. 동일한 백 0 1 착수는 한 상태에서 합법이고 다른 상태에서 패로 거부된다. 전수검증에서 얻은 상태조건이 실제 대국 코드에 이어졌음을 확인한다.

5×5 색상 교대 벤치마크

WRRR MCTS와 통제 알고리즘을 흑백으로 번갈아 배치했다. 각 수는 48 simulations를 사용했다. 균등 MCTS는 같은 트리탐색과 같은 계산예산을 사용하지만 Renderer prior와 guided rollout을 제거했다. 무작위 통제군은 합법 착수 가운데 하나를 고른다.

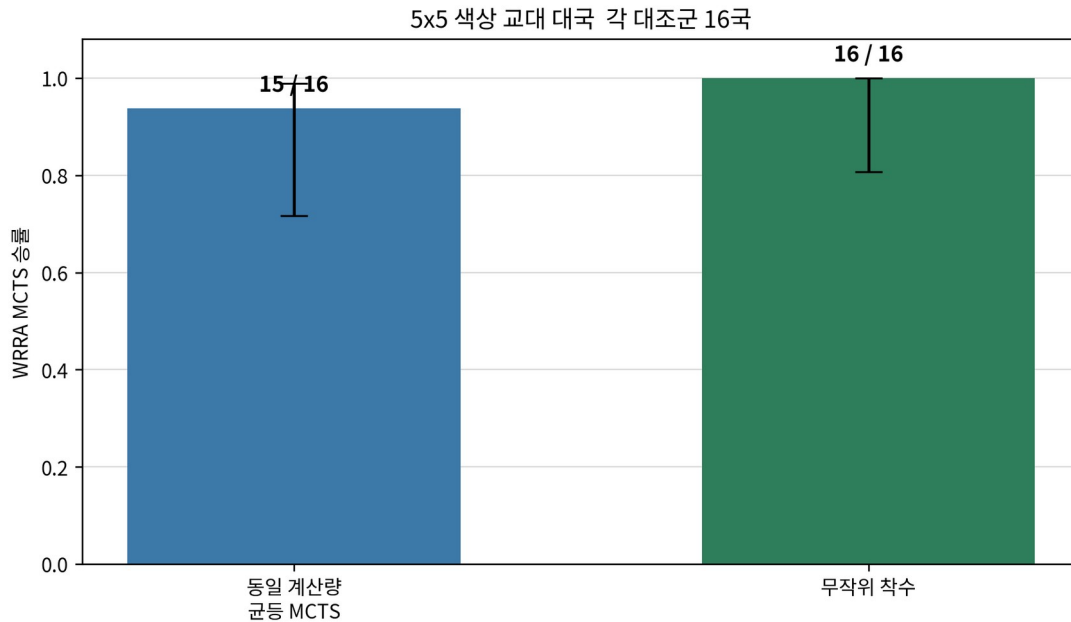


그림 7 동일 규칙과 색상 교대 조건에서의 WRRR MCTS 대조 대국

대조군	대국	WRRR 승패	승률	Wilson 95% 구간	평균 수수
균등 MCTS	16	15승 1패	93.8%	71.7-98.9%	34.8
무작위	16	16승 0패	100.0%	80.6-100.0%	37.9

이 시험으로 확정되는 것은 고정 Renderer가 실제 수 선택에 유효한 편향을 주었고, 같은 계산예산의 균등 탐색보다 이 표본에서 더 많은 대국을 이겼다는 사실이다. 전문 엔진보다 강하다는 결론이나 19×19 기력의 일반화는 이 표본에서 꺼내지 않는다. 다음 버전은 판 크기와 seed를 늘리고 GTP 기반 외부 엔진 대국을 추가하면 된다.

9×9 자기대국

WRRR MCTS 두 개가 각각 24 simulations로 둔 9×9 자기대국은 82수 뒤 two_passes로 끝났다. 흑 면적 38, 백 면적 41, 덤 5.5를 적용한 흑 기준 결과는 -8.5집이며 승자는 W이다.

9×9 WRRRA 자기대국 최종 판

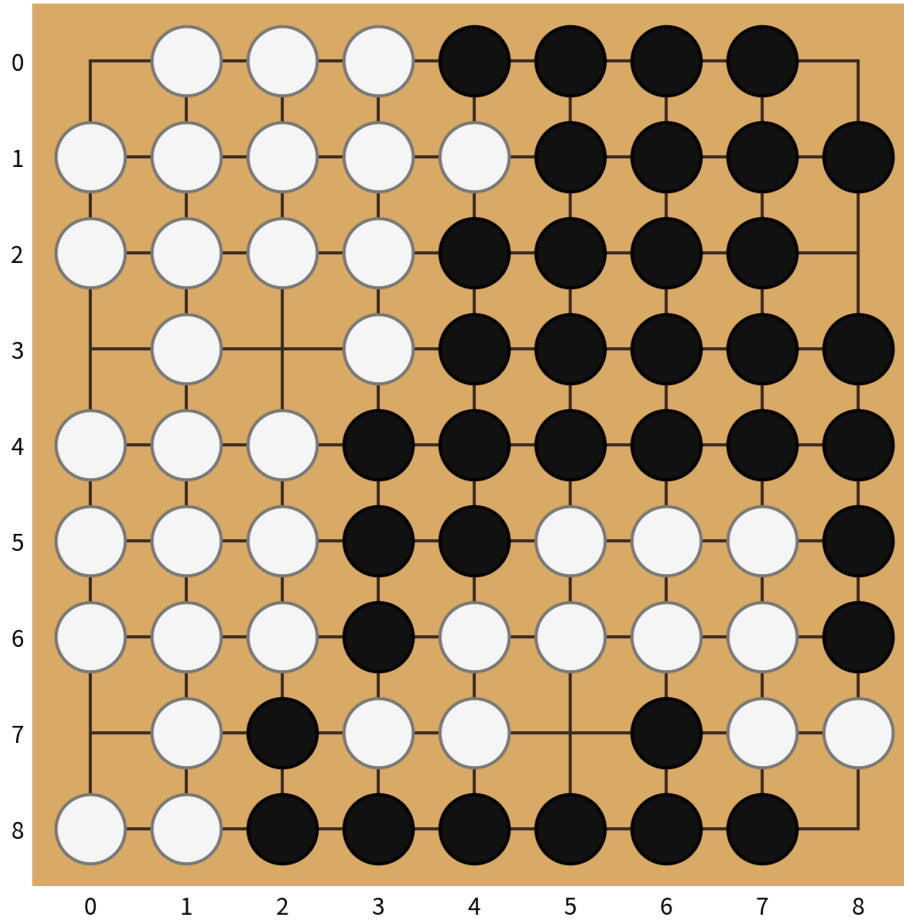


그림 8 9×9 WRRRA 자기대국의 최종 판 SGF 기보를 별도 제공한다

9×9 자기대국 첫 수의 후보 장부

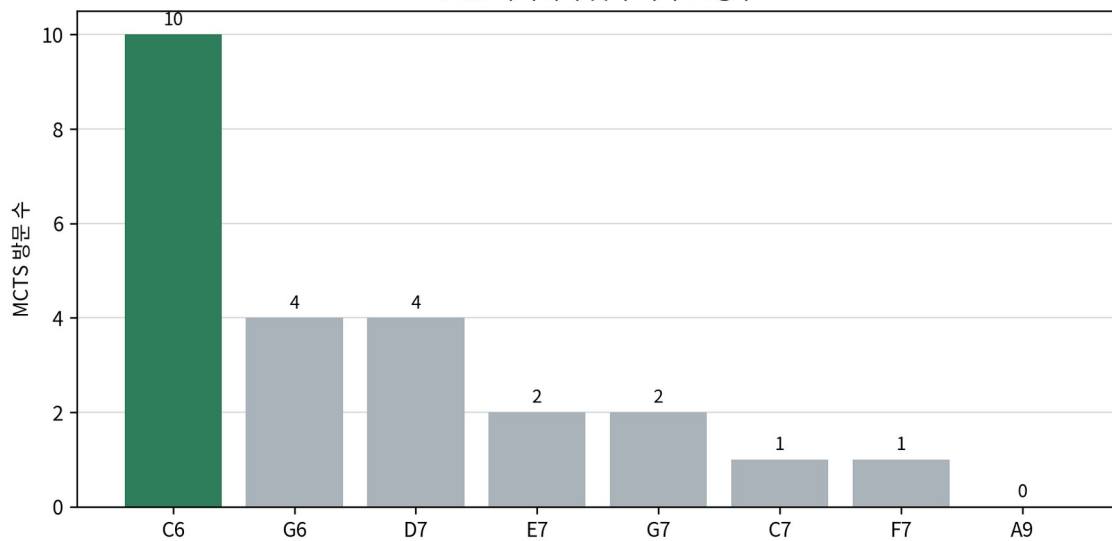


그림 9 첫 착수에서 실제로 남긴 후보별 방문 장부

후보	방문	평균값	prior	Rendererer 점수	phenotype
C6	10	+0.000	0.0142	+1.080	extension
G6	4	+0.500	0.0142	+1.080	extension
D7	4	+0.000	0.0142	+1.080	extension
E7	2	+0.000	0.0142	+1.080	extension
G7	2	+0.000	0.0142	+1.080	extension
C7	1	-1.000	0.0142	+1.080	extension
F7	1	-1.000	0.0142	+1.080	extension
A9	0	+0.000	0.0092	+0.493	extension

전체 자기대국 기보

수순	흑	백
1	C6	G6
2	F7	D7
3	D3	C7
4	F8	F3
5	E7	D8
6	G7	B7
7	G8	B6
8	H7	F4
9	H8	G4
10	E6	D6

수순	흑	백
11	F6	G3
12	E5	E3
13	E4	C5
14	G5	B5
15	G9	B8
16	H6	E2
17	D4	H4
18	F5	B4
19	E9	E8
20	D5	H3
21	C2	D2
22	C1	C8
23	J6	J9
24	H9	H2
25	J5	C9
26	H5	D9
27	D1	B3
28	F1	A7
29	E1	C3
30	G1	A4
31	J8	B2

수순	흑	백
32	J4	B1
33	A2	F9
34	J3	A1
35	H1	A3
36	E9	A5
37	F9	J2
38	G2	C4
39	A9	B9
40	G6	A8
41	pass	pass

SGF는 같은 수순을 표준 좌표로 저장한다. JSON Ledger에는 각 수의 상위 후보, 방문 수, 평균값, prior, Renderer 점수, phenotype과 계산시간이 포함된다. 따라서 최종 착수뿐 아니라 왜 그 수가 선택되었는지 다시 검사할 수 있다.

6 이 연구의 학문적 가치

하나의 추상어가 아니라 연결된 검증 사슬

1. 같은 Core 계약을 서로 다른 게임에 적용하되 필요한 residue의 차이를 보존했다. 공통성은 차이를 지우는 통합이 아니라 같은 검사법을 공유하는 통합이다.
2. 구성적 사례에서 시작해 전체 도달그래프의 반례와 값 보존으로 확장했다. 설명이 계산결과에 의해 수정될 수 있는 구조를 갖췄다.
3. 상태의 정확성과 탐색의 효율성을 분리했다. residue 제거는 합법성을 바꾸고 Renderer 제거는 값은 유지한 채 계산량을 늘린다.
4. 완전탐색이 불가능한 판에서는 검증된 전이기를 버리지 않고 근사탐색과 결합했다. 작은 판의 exact proof와 큰 판의 bounded action을 연결했다.
5. Renderer의 채널과 가중치, 후보별 방문 수를 공개해 숨은 판단을 줄였다. 각 착수의 구조적 근거를 phenotype으로 되짚을 수 있다.
6. 오목 바둑 장기 체스 같은 서로 다른 게임을 state law residue renderer ledger라는 동일 인터페이스에 올릴 수 있는 재사용 가능한 연구도구를 만들었다.

새롭게 드러난 장점

장점	이번 연구의 증거	다른 복잡계로의 의미
상태 aliasing 자동탐지	752개 residue 의존 동치류	같은 관측 뒤 다른 허용행동 탐색
증명형 후보축약	Connect K 전 상태 값 불일치 0	국소 phenotype을 안전한 계산인증서로 사용
정확성과 속도의 분리	residue와 Renderer 제거시험	모델 기억과 계산예산을 별도 설계
축척 전환	3×3 exact에서 9×9 MCTS로 이동	전수검증과 제한계산의 연결
설명가능한 행동	11개 채널과 후보 Ledger	결과뿐 아니라 선택근거 재검사
Core 불변성	0.1 0.2 Go 0.1 모두 Core 수정 0	응용이 늘어도 상위 계약 유지

특히 작은 완전계에서 상태표현과 축약규칙을 검증한 뒤 같은 코드를 실제 행위자에 넣는 순서는 중요하다. 기력이 먼저가 아니라 규칙의 정확성, 상태의 충분성, 축약의 무손실성, 제한계산의 성능을 차례로 분리한다. 이 순서는 전력망, 생물학적 조절, 에이전트 행동처럼 전체공간을 열거할 수 없는 영역에서도 작은 검증계와 실제 운영계를 연결하는 방법으로 확장할 수 있다.

현재 성과와 다음 검증을 구분하는 기준

현재 확정	다음에 확장
단순 패를 포함한 합법수와 상태전이	positional superko의 압축 residue
5×5 통제 대국과 9×9 실제 실행	9×9 다중 seed 대회와 19×19 시간제한 대국
고정된 투명 Renderer 채널	가중치 ablation과 자동보정
MCTS 결합의 실행 가능성	KataGo 등 외부 GTP 엔진과 등급화
area score와 SGF JSON 장부	사석 판정과 중국식 일본식 규칙 profile 분리

이 구분은 현재 성과를 낮추기 위한 문장이 아니라 다음 실험의 입력과 출력이 무엇인지 고정하는 장부다. 현재 버전은 이미 사람이 둘 수 있는 독립 프로그램이며, 전문 대국 성능은 같은 실행계약 위에서 계산예산과 평가함수를 확장해 측정할 후속 항목이다.

7 재현과 Zenodo 공개 구성

파일 계보

파일	역할	SHA 256
wrra_game_0_1.py	구성적 오목과 바둑 분석	e5bdf92a69b7bd50c7444d099cbb5c0848fd7beeb2bead2d78d5337825d36bd6c
wrra_game_0_1_results.json	0.1 결과 장부	4de1c203b65a0b85dd71cc2e2988d28befedbbdc2900e25a713383e86d3c2dc4
wrra_game_0_2.py	Connect K와 3×3 바둑 전수검사	ec010ec0de5f045dcf47956a9cfd223ded06a12a5933a6c51ffacfb4af95e2b
wrra_game_0_2_results.json	0.2 결과 장부	bed55e8efda6f8972be0cecc04f929d900fd4292278fdce82262500bc0abbab8
wrra_go_0_1.py	실제 대국 엔진과 통제시험	8549c63f35fbf13d6c4d0ae48ed797132bfca8edf69177d84d35cf3d4f6328c9
wrra_go_0_1_benchmark.json	색상 교대 벤치마크	1368144e9df8a02ef391b4ddb8ed9c1a39eb45a46dba579afc5a4a162d20b98a
wrra_go_0_1_rule_tests.json	결정적 규칙 회귀검사	c1e6c40a6f7803285b9ab0d45f493cc136ff648b62cb9d6b7809a433ea33e376
wrra_go_0_1_selfplay_9x9.json	9×9 자기대국 전체 Ledger	fa340a48ed25887b3bc54cc86f48df9531bfc12e3f76c5a11ace9ab101093195

파일	역할	SHA 256
wrra_go_0_1_selfplay_9x9.sgf	9×9 자기대국 SGF	6d1c86ac405e54f514c9742d93a194fc0049fc10b63b6a30219c1564df297680

재현 순서

1. python wrra_go_0_1.py test를 실행해 열 개 규칙검사가 모두 true인지 확인한다.
2. python wrra_game_0_2.py를 실행해 Connect K 6,036,001상태와 값 불일치 0을 확인한다.
3. 같은 결과에서 3×3 바둑 132,161상태와 residue 의존 동치류 752개를 확인한다.
4. python wrra_go_0_1.py benchmark --size 5 --paired-games 8 --simulations 48을 실행한다.
5. python wrra_go_0_1.py selfplay --size 9 --simulations 24를 실행해 JSON과 SGF를 만든다.
6. 사람 대 엔진은 python wrra_go_0_1.py play --size 9 --human B로 시작한다.

이번 벤치마크의 기록된 실행시간은 137.78초이며 환경에 따라 달라진다. 상태 수, 합법성 검사, seed와 대국 결과는 같은 Python 동작과 입력 조건에서 재현된다. 0.2의 완전탐색 결과는 기존 JSON을 포함했으므로 전체 재계산 없이도 장부를 검사할 수 있다.

Zenodo 권장 메타데이터

필드	권장 입력
Title	WRRa Game 1.0 Minimal Sufficient State Exhaustive Validation and a Playable Go Engine
Subtitle	From Omok and Go State Analysis to WRRa Go 0.1
Creator	Wonsik Choi
Contact	janefather@gmail.com
Resource type	Software and technical report
Language	Korean with English metadata

필드	권장 입력
Keywords	WRRR, Go, Baduk, Omok, Connect K, MCTS, state sufficiency, residue, renderer, exhaustive search, complex systems
Related identifier	WRRR Core 1.0 https://doi.org/10.5281/zenodo.22650956
License	보고서·문서·데이터: CC BY 4.0; 소스 코드: MIT

Description

This release integrates the WRRR Game research program from constructive game analysis to exhaustive finite-state validation and a playable Go engine. WRRR Game 0.1 identifies an Omok forced-win phenotype and a simple-ko residue counterexample. WRRR Game 0.2 exhaustively enumerates 6,036,001 reachable Connect-K states and 132,161 reachable 3×3 Go states, finding zero minimax-value mismatches for the certified Renderer reduction and 752 visible-state classes whose legal actions depend on the previous-board residue. WRRR Go 0.1 reuses the verified state transition system in a playable 9×9 engine, combining a transparent eleven-channel Renderer with Monte Carlo Tree Search. The archive includes source code, machine-readable ledgers, deterministic rule tests, paired-control matches, and an SGF self-play record. The work evaluates rule-permitted continuations; it does not describe the residue or search as prediction of a pre-existing future.

참조 계보

WRRR Core 1.0 Frozen Domain Agnostic Execution Architecture DOI 10.5281/zenodo.22650956

WRRR Game 0.1 오목과 바둑에서 단순 규칙이 만드는 복잡성 2026년 9월 16일

WRRR Game 0.2 오목형 연결게임 완전탐색과 바둑 residue 상태충분성 검증 2026년 9월 17일

WRRR Go 0.1 본 보고서에 포함된 실제 대국 엔진 2026년 9월 17일

최종 판정

이 연구는 단순규칙이 복잡한 게임을 만든다는 설명에서 멈추지 않았다. 오목에서는 Renderer가 어떤 phenotype을 안전한 계산축약으로 바꿀 수 있는지 전체 상태에서 검사했고, 바둑에서는 어떤 residue가 합법수 판정을 위해 반드시 상태에 들어가야 하는지 전체 도달그래프의 반례로 확정했다. 이어 같은 전이기를 9×9 대국 엔진에 넣어 실제 수를 선택하고 SGF와 후보 Ledger를 남겼다.

따라서 WRRRA Game 1.0의 핵심 성과는 한 문장으로 정리된다. 최소 충분상태로 규칙의 정확성을 지키고, Renderer로 계산해야 할 구조를 드러내며, 제한된 탐색으로 실제 행동을 선택하는 하나의 재현 가능한 게임 실행계를 구축했다.